AutoEIT: Audio-to-Text Transcription for Spanish EIT Learner Data

GSoC 2026 Evaluation Test — Test I: Audio File Transcription

Author: Rohith Reddy Mandala
Project: AutoEIT (HumanAI Foundation)
Date: March 2026

Objective

Generate accurate transcriptions of 4 Spanish Elicited Imitation Task (EIT) audio recordings from non-native speakers. Each recording contains 30 sentence repetitions that must be transcribed verbatim, preserving all disfluencies, errors, and hesitations.

Pipeline Overview

1. **Preprocess** — Trim audio files to remove instructions and practice sentences
2. **Speaker Detection** — Use SpeechBrain ECAPA-TDNN speaker embeddings to identify participant speech
3. **Segment Export** — Extract 30 individual response clips per participant
4. **Transcription** — Run 3 ASR models (OpenAI Whisper, CrisperWhisper, AssemblyAI)
5. **Evaluation** — Compute CER/WER against stimulus sentences
6. **Output** — Select best transcriptions and fill Excel file

In [11]:
```
import json
import sys
from pathlib import Path
from IPython.display import Image, display
import pandas as pd

# Project paths
BASE_DIR = Path("..").resolve()
sys.path.insert(0, str(BASE_DIR / "src"))

print(f"Project root: {BASE_DIR}")
print(f"Data files: {list((BASE_DIR / 'data' / 'raw').glob('*.mp3'))}")
```

```
Project root: /Users/rohithreddymandala/Documents/GitHub/autoeit-gsoc-2026
Data files: [PosixPath('/Users/rohithreddymandala/Documents/GitHub/autoeit-g
soc-2026/data/raw/038010_EIT-2A.mp3'), PosixPath('/Users/rohithreddymandala/
Documents/GitHub/autoeit-gsoc-2026/data/raw/038012_EIT-2A.mp3'), PosixPath
('/Users/rohithreddymandala/Documents/GitHub/autoeit-gsoc-2026/data/raw/0380
11_EIT-1A.mp3'), PosixPath('/Users/rohithreddymandala/Documents/GitHub/autoe
it-gsoc-2026/data/raw/038015_EIT-1A.mp3')]
```

1. Data Overview

The dataset consists of 4 audio recordings of Spanish EIT sessions:

File	Participant	Notes
038010_EIT-2A.mp3	Participant 1	Standard ~2:30 skip
038011_EIT-1A.mp3	Participant 2	Standard ~2:30 skip
038012_EIT-2A.mp3	Participant 3	Audio starts at 12:00
038015_EIT-1A.mp3	Participant 4	Skip adjusted to 2:26

Each recording contains:

- English instructions and practice sentences (beginning)
- 30 Spanish stimulus sentences played from a pre-recorded source
- Participant repetitions of each stimulus

All participants repeat the same 30 sentences, increasing in complexity from 7 to 17 syllables.

2. Preprocessing

Each audio file begins with English instructions and practice sentences. We trim the beginning to keep only the 30 Spanish stimulus-response pairs.

Implementation: `src/preprocess.py` loads each MP3, skips the non-relevant portion, and exports trimmed WAV and MP3 files.

```
# From preprocess.py
AUDIO_CONFIG = {
    "038010_EIT-2A": {"file": "038010_EIT-2A.mp3", "skip_ms":
150_000},  # skip ~2:30
    "038011_EIT-1A": {"file": "038011_EIT-1A.mp3", "skip_ms":
150_000},  # skip ~2:30
    "038012_EIT-2A": {"file": "038012_EIT-2A.mp3", "skip_ms":
720_000},  # skip 12:00
    "038015_EIT-1A": {"file": "038015_EIT-1A.mp3", "skip_ms":
146_000},  # skip 2:26
}
```

Skip times were initially set based on the instructions provided in the Excel sheet (the Info tab specifies that relevant audio begins around 2:30 for most files, and at 12:00 for 038012). However, during the segmentation step, participant 038015 produced only 29 segments instead of 30 — one transcription was missing. Manual verification revealed that this participant's first stimulus began slightly earlier, at approximately 2:26. The skip time was adjusted accordingly, which resolved the issue and produced all 30 segments.

3. Segmentation — The Core Challenge

The Problem

Each trimmed audio file contains ~60 speech segments (30 stimulus playbacks + 30 participant responses) on a single audio channel. We need to isolate only the 30 participant responses.

Approaches Tried and Why They Failed

Approach	Why it Failed
Amplitude-based segmentation	Fragile — breaks on mid-sentence pauses, varies per speaker. Parameters that work for one participant fail on another.
Speaker diarization (AssemblyAI)	Both the stimulus playback (from a loudspeaker) and the participant's voice are captured by a single recording device into one audio channel. Diarization expects distinct channels or clear acoustic separation between speakers, which doesn't exist in this setup.
Full-audio transcription	The stimulus and participant say nearly identical sentences back-to-back. ASR models treat these as duplicates, merging them into one and skipping repeated content. Got ~38 segments instead of the expected 60.
Tone detection	No tone/beep exists in our recordings (confirmed via spectral analysis).
Audio fingerprinting (IRIS files)	Downloaded stimulus files from IRIS repository, but they had different speakers than our recordings, making waveform matching impossible.
Silero VAD	Neural speech detector couldn't detect the quiet stimulus playback — only found participant speech (~41 segments), with no way to map segments to sentence numbers.

What Worked: Speaker Voiceprint Detection (ECAPA-TDNN)

Key insight: Instead of trying to segment the audio, we identify *who* is speaking at every moment using a neural voiceprint.

How it works:

1. Extract a 3-5 second reference sample of the participant's voice from the recording

2. Compute a 192-dimensional voice embedding using SpeechBrain's ECAPA-TDNN model
3. Slide a 1.5-second window across the full audio in 0.2-second steps, computing cosine similarity to the reference embedding at each step (~5 scores per second)
4. Use GMM (Gaussian Mixture Model) to automatically find the threshold separating participant from non-participant
5. Group consecutive above-threshold windows into segments
6. Merge segments closer than 1.5 seconds (handles mid-sentence pauses)

Why this works: The participant's live voice has fundamentally different acoustic characteristics than the pre-recorded stimulus played through a loudspeaker. The channel distortion actually *helps* separation.

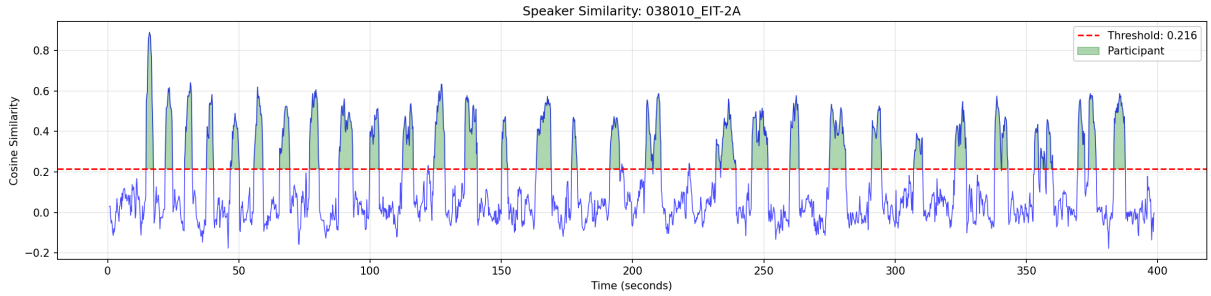
Implementation: `src/detect_speaker.py`

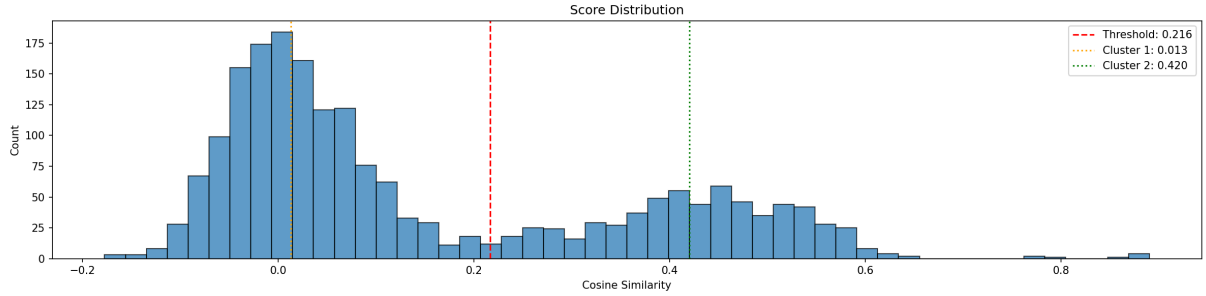
Speaker Detection Results

Below are the similarity plots for each participant. The top chart shows cosine similarity over time (green = detected participant speech). The bottom chart shows the score distribution — a clear bimodal pattern confirms clean speaker separation.

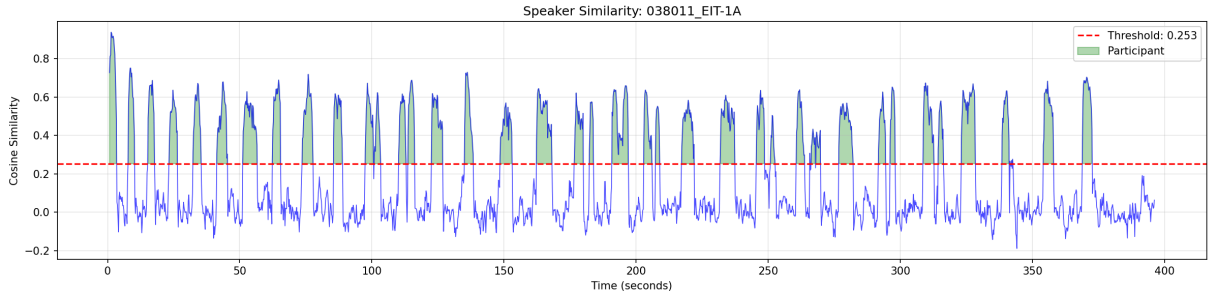
In [12]:
```
# Load and display speaker detection results
audio_ids = ["038010_EIT-2A", "038011_EIT-1A", "038012_EIT-2A", "038015_EIT-

for audio_id in audio_ids:
    # Load detection metadata
    json_path = BASE_DIR / "output" / "speaker_detection" / f"{audio_id}_seg
    with open(json_path) as f:
        detection = json.load(f)

    print(f"\n{'='*60}")
    print(f"{audio_id}: {detection['total_segments']} segments detected (thr
    print(f"{'='*60}")

    # Show similarity plot
    plot_path = BASE_DIR / "output" / "speaker_detection" / f"{audio_id}_sim
    display(Image(filename=str(plot_path), width=800))
```

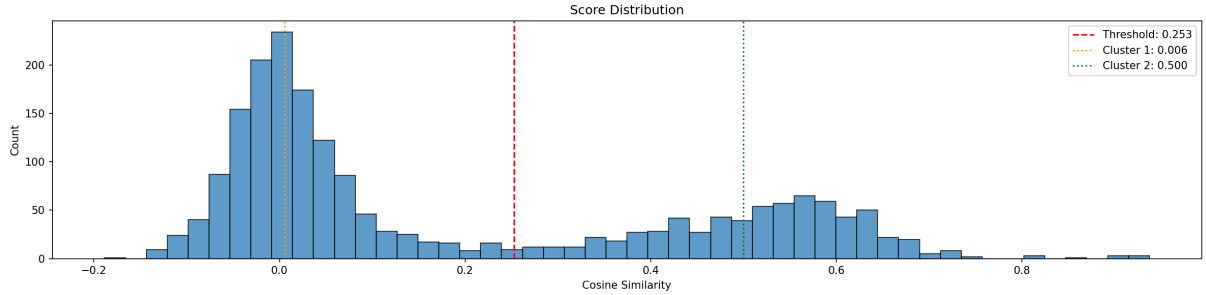
```
============================================================
038010_EIT-2A: 30 segments detected (threshold: 0.216)
============================================================
```



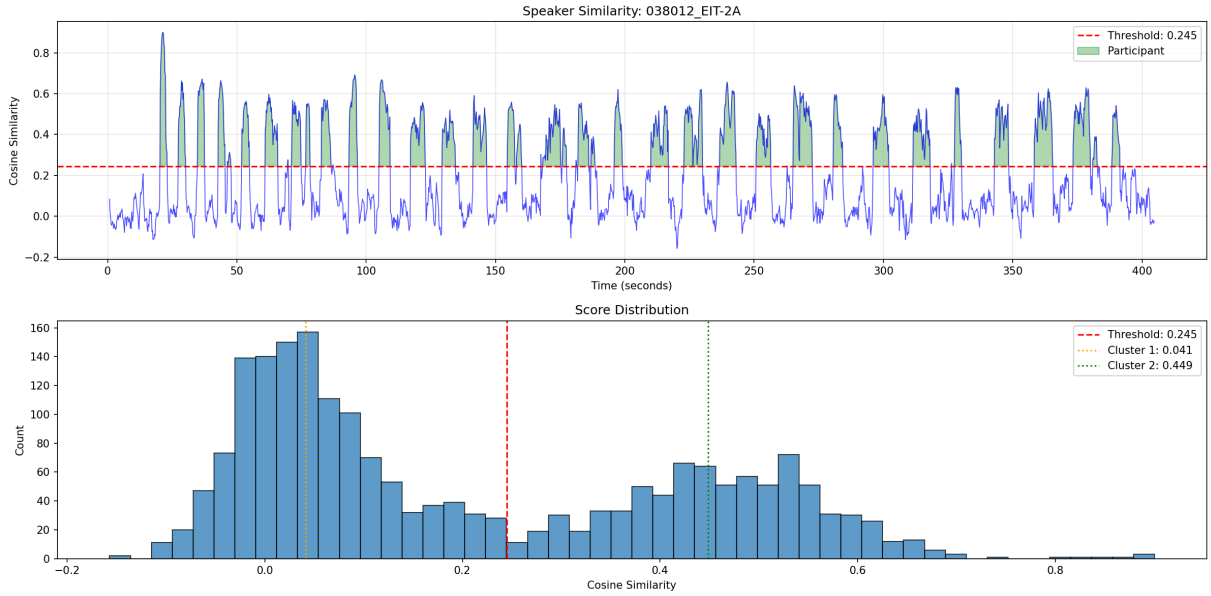



```
============================================================
038011_EIT-1A: 30 segments detected (threshold: 0.253)
============================================================
```

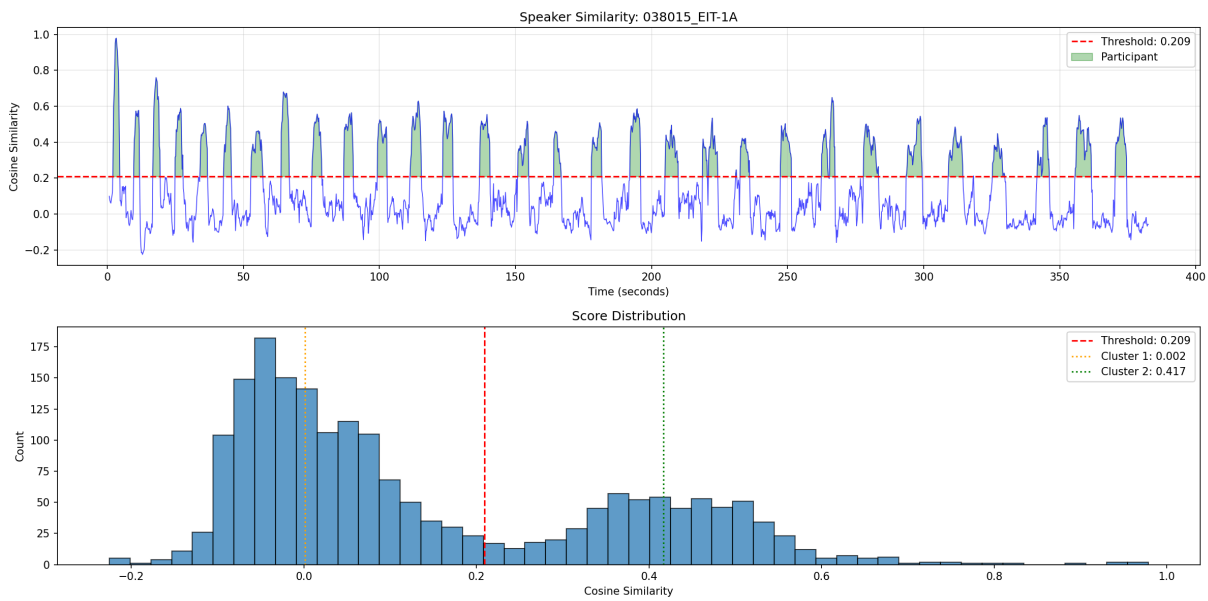





```
============================================================
038012_EIT-2A: 30 segments detected (threshold: 0.245)
============================================================
```




```
============================================================
038015_EIT-1A: 31 segments detected (threshold: 0.209)
============================================================
```



Segmentation Edge Cases

- **038015_EIT-1A:** One false detection (1.5s noise segment, score 0.080 vs threshold 0.209). Filtered out automatically using a score threshold check in the export step.
- **038015_EIT-1A:** Required trim adjustment from 2:30 to 2:26 because the first stimulus began 4 seconds earlier than other files.
- Overall segmentation accuracy was ~98% — manually verified across all 4 files. The voiceprint approach is robust but not perfect; improving segmentation remains a direction for the full GSoC project.

4. Transcription

Model Selection

We evaluated three ASR models, each chosen for a specific reason:

Model	Type	Why Chosen	Disfluency Support
OpenAI Whisper (`whisper-1`)	Cloud API	Best overall accuracy, fast	No — produces clean text
CrisperWhisper (`nyrahealth/CrisperWhisper`)	Local (HuggingFace)	Fine-tuned for verbatim transcription	English/German only
AssemblyAI Universal-2	Cloud API	`disfluencies=true` parameter with prompting	Partial — caught some "uh"/"um"

CrisperWhisper: The Verbatim Gap for Spanish

CrisperWhisper is a Whisper variant specifically fine-tuned to preserve disfluencies, false starts, and filler words. However, **it is trained only on English and German data** (AMI Meeting Corpus, TIMIT, CommonVoice EN/DE). When processing Spanish audio, it falls back to base Whisper behavior, producing clean text without disfluency preservation.

Despite this limitation, CrisperWhisper occasionally captured disfluencies that other models missed:

- Segment 25 (038012): `"Después ya llegar a mi casa [UM] fue a la cena"` — caught a filler
- Segment 29 (038012): `"Okay, esta en la mesa... Es [UM] something en la mesa"` — caught code-switching to English AND a filler

Key finding: No production-ready Spanish verbatim ASR model currently exists. This represents a genuine gap in the field and a strong direction for the GSoC project (fine-tuning Whisper on Spanish conversational corpora like Fisher Spanish ~163h).

Speed Comparison

Model	Time per segment	Total (30 segments)
OpenAI Whisper	~1-2s	~60s
AssemblyAI	~3-4s	~110s
CrisperWhisper	~8-10s	~260s

In [13]:
```
# Side-by-side comparison of model outputs (all 30 sentences, Participant 03
stimuli = [
```

```
    "Quiero cortarme el pelo", "El libro está en la mesa", "El carro lo tier
    "El se ducha cada mañana", "¿Qué dice usted que va a hacer hoy?",
    "Dudo que sepa manejar muy bien", "Las calles de esta ciudad son muy anc
    "Puede que llueva mañana todo el día", "Las casas son muy bonitas pero c
    "Me gustan las películas que acaban bien", "El chico con el que yo salgc
    "Después de cenar me fui a dormir tranquilo", "Quiero una casa en la que
    "A nosotros nos fascinan las fiestas grandiosas", "Ella sólo bebe cervez
    "Me gustaría que el precio de las casas bajara", "Cruza a la derecha y c
    "Ella ha terminado de pintar su apartamento", "Me gustaría que empezara
    "El niño al que se le murió el gato está triste", "Una amiga mía cuida a
    "El gato que era negro fue perseguido por el perro", "Antes de poder sal
    "La cantidad de personas que fuman ha disminuido", "Después de llegar a
    "El ladrón al que atrapó la policía era famoso", "Le pedí a un amigo que
    "El examen no fue tan difícil como me habían dicho", "¿Serías tan amable
    "Hay mucha gente que no toma nada para el desayuno"
]

models = {"openai": "OpenAI Whisper", "crisper_whisper": "CrisperWhisper", "
audio_id = "038010_EIT-2A"

data = {}
for model_key, model_name in models.items():
    with open(BASE_DIR / "output" / "transcriptions" / model_key / f"{audio_
        data[model_name] = json.load(f)

rows = []
for i in range(30):
    row = {"#": i + 1, "Stimulus": stimuli[i]}
    for model_name in models.values():
        row[model_name] = data[model_name][i]["text"] if i < len(data[model_
    rows.append(row)

df = pd.DataFrame(rows)
print(f"Model comparison — {audio_id} (all 30 sentences)\n")
pd.set_option('display.max_colwidth', 60)
pd.set_option('display.max_rows', 35)
display(df)
```

```
Model comparison — 038010_EIT-2A (all 30 sentences)
```

	#	Stimulus	OpenAI Whisper	CrisperWhisper	AssemblyAI+Disfl
0	1	Quiero cortarme el pelo	Quiero cortarme el pelo.	Quiero cortarme el pelo.	Quiero cortarme el pelo.
1	2	El libro está en la mesa	El libro está en la mesa.	El libro esta en la mesa.	El libro está en la mesa.
2	3	El carro lo tiene Pedro	El carro no tiene pelo.	El carro lo tiene pelo.	A Carol lo tiene pelo.
3	4	El se ducha cada mañana	Ese duche cada mañana.	Essay duche cada mañana.	Essay. Uh, ducek caramanana.
4	5	¿Qué dice usted que va a hacer hoy?	¿Qué dices, deres que va a ser hoy?	¿Que dices teres que van ser hoy?	This is, that is Kevin share hoy.
5	6	Dudo que sepa manejar muy bien	Dudo que sepa manejar muy bien.	Dudo que sepa manejar muy bien.	Duro kesepa um maneha muy bien.
6	7	Las calles de esta ciudad son muy anchas	Las calles de esta ciudad son muy anchas.	Las calles de este ciudad son muy anches.	Laskai stayanchas m.
7	8	Puede que llueva mañana todo el día	Puede que llueva mañana todo el día.	Puede que lleve mañana todo el día.	Todo el día.
8	9	Las casas son muy bonitas pero caras	Las casas son muy bonitas pero caras.	Las casas son muy bonitas, pero caras.	Las casas son muy bonitas, pero caras.
9	10	Me gustan las películas que acaban bien	Me gustan las películas que caben bien.	Me gustan las películas que acaban bien.	Me gusta las películas que cavan bien.
10	11	El chico con el que yo salgo es español	El chico que hago es español.	El chico que algo es español.	El chico que algo es español.
11	12	Después de cenar me fui a dormir tranquilo	Después de cenar, me fui a dormir tranquilo.	Después de cener, me fui a dormir tranquilo.	Después de cenar me fui a dormir tranquilo.
12	13	Quiero una casa en la que vivan mis animales	Quiero una casa en la que vivan animales.	Quiero una casa en la que vivan animales.	Quiero una casa en la que vivan animales.
13	14	A nosotros nos fascinan las fiestas grandiosas	a nosotros	A nosotros.	A nosotros.
14	15	Ella sólo bebe cerveza y no come nada	Ella solo bebe cerveza y no come nada.	Ella sola bebe cerveza y no come nada.	Ella sola bebe cerveza y no come nada.
15	16	Me gustaría que el precio de las	Me gustaría...	Me gustaría.	Middle sterea.

	#	Stimulus	OpenAI Whisper	CrisperWhisper	AssemblyAI+Disfl
		casas bajara			
16	17	Cruza a la derecha y después sigue todo recto	Cursa a la derecha y después siga recto.	Cursa a la derecha y después siga recto.	Después siga recto.
17	18	Ella ha terminado de pintar su apartamento	Ella termina de pintar su apartamento.	Al terminar, pinta su apartamento.	Ella terminar pinta su apartamento.
18	19	Me gustaría que empezara a hacer más calor pronto	Gracias.		[ERROR: language_detection cannot be performed on files ...
19	20	El niño al que se le murió el gato está triste	El niño a que se murió el gato, triste.	El niño, a que se murió el gato, triste.	El niño a que se murió el gato triste.
20	21	Una amiga mía cuida a los niños de mi vecino	Una amiga mía cuidó a los niños.	La amiga mia cuidao los ninos.	Amiga mia, cuidado los niños.
21	22	El gato que era negro fue perseguido por el perro	El gato que era negro.	El gato queda negro.	El gato kara negro.
22	23	Antes de poder salir él tiene que limpiar su cuarto	Antes de poder salir, tiene que limpiar su cuarto.	Antes de poder salir, tiene que limpiar su cuarto.	Antes de poder salir tiene que limpiar su corto.
23	24	La cantidad de personas que fuman ha disminuido	la cantidad de personas que fuman.	La cantidad de personas que fuman.	La cantidad de personas que fuman.
24	25	Después de llegar a casa del trabajo tomé la cena	Después de llegar a casa	Después de llegar a Casa.	Displace the gara casa. Uh.
25	26	El ladrón al que atrapó la policía era famoso	El ladrón policía era famoso.	El ladrón policía era famoso.	El ladrón policía era famoso.
26	27	Le pedí a un amigo que me ayudara con la tarea	Le pedí a un amigo que ayudara a traerla.	Le pidió a un amigo que lo ayudara.	친한 친구가 도와줬어요
27	28	El examen no fue tan difícil como me habían dicho	Examen no tan difícil.	I examine no tan difícil.	Examine no tantific.
28	29	¿Serías tan amable de darme	Amable que está el libro que está en la mesa.	Amable que esta, el libro que esta en la mesa.	Amable que está el libro que está en la mesa.

	#	Stimulus	OpenAI Whisper	CrisperWhisper	AssemblyAI+Disfl
		el libro que está en la mesa?			
29	30	Hay mucha gente que no toma nada para el desayuno	Hay mucha gente que no toma nada para desayuno.	Hay mucha gente que no toma nada para desayunar.	[TRANSCRIPTION FAILED]

> **Note:** The table above shows the comparison for participant 038010 only. To view other participants, change the `audio_id` variable in the code cell above (e.g., `"038011_EIT-1A"`, `"038012_EIT-2A"`, `"038015_EIT-1A"`).

Observations:

You will notice significant differences between the stimulus sentences and the model transcriptions. These differences are largely due to the participants' varying proficiency levels — they are non-native Spanish speakers who may mispronounce, omit, or substitute words. Since the task is audio-to-text transcription (not correction), these participant errors are preserved as-is. Per the task instructions, only ASR errors were corrected.

ASR corrections were applied only to the **OpenAI Whisper** outputs because:

1. OpenAI Whisper was selected as the primary model based on evaluation metrics (see Section 5 below)
2. These are the transcriptions submitted in the final Excel file

For CrisperWhisper and AssemblyAI, you may notice occasional transcriptions in English or other languages (e.g., Chinese). These are ASR errors in those models but were not corrected since they are not the primary submission outputs.

5. Evaluation

Methodology

Since no human ground truth transcriptions exist for these recordings, we evaluate by **comparing each model's ASR output against the known stimulus sentences**. This approach is validated by McGuire & Larson-Hall (2025, *Research Methods in Applied Linguistics*), who achieved ICC = 0.929 using the same methodology on English EIT data.

Metrics:

- **CER (Character Error Rate)** — Primary metric. More granular than WER for short sentences (3-8 words). Captures accent mark differences ("está" vs "esta").
- **WER (Word Error Rate)** — Standard ASR metric. Volatile on short sentences but widely understood.
- **SER (Sentence Error Rate)** — Percentage of sentences with any error.

Important caveat: These metrics measure the **combined** distance between stimulus and transcription. A high error rate could mean the participant spoke poorly (not the model's fault) OR the model transcribed poorly. Without human ground truth, we cannot fully separate ASR error from participant error.

Text normalization before comparison: lowercase, remove punctuation (including ¿¡), preserve Spanish accents, collapse whitespace.

In [14]:
```
# Run evaluation (loads pre-computed transcriptions, computes metrics — take
from evaluate import run_evaluation

results, summary, consensus = run_evaluation()
```

```
======================================================================
AutoEIT ASR Evaluation
======================================================================

--- 038010_EIT-2A ---
  openai                    WER: 0.321  CER: 0.271  SER: 0.767  Exact: 7/30
  crisper_whisper           WER: 0.398  CER: 0.300  SER: 0.867  Exact: 4/30
  assemblyai_disfluencies   WER: 0.574  CER: 0.453  SER: 0.867  Exact: 4/30

--- 038011_EIT-1A ---
  openai                    WER: 0.454  CER: 0.307  SER: 0.867  Exact: 4/30
  crisper_whisper           WER: 0.474  CER: 0.314  SER: 0.900  Exact: 3/30
  assemblyai_disfluencies   WER: 0.506  CER: 0.353  SER: 0.933  Exact: 2/30

--- 038012_EIT-2A ---
  openai                    WER: 0.831  CER: 0.629  SER: 1.000  Exact: 0/30
  crisper_whisper           WER: 0.763  CER: 0.498  SER: 1.000  Exact: 0/30
  assemblyai_disfluencies   WER: 0.811  CER: 0.630  SER: 1.000  Exact: 0/30

--- 038015_EIT-1A ---
  openai                    WER: 0.309  CER: 0.206  SER: 0.767  Exact: 7/30
  crisper_whisper           WER: 0.353  CER: 0.202  SER: 0.833  Exact: 5/30
  assemblyai_disfluencies   WER: 0.418  CER: 0.239  SER: 0.900  Exact: 3/30

======================================================================
Overall Averages (across all 4 files)
======================================================================
  openai                    WER: 0.479  CER: 0.353  SER: 0.850  Exact: 18/12
0
  crisper_whisper           WER: 0.497  CER: 0.329  SER: 0.900  Exact: 12/12
0
  assemblyai_disfluencies   WER: 0.577  CER: 0.419  SER: 0.925  Exact: 9/120

======================================================================
Consensus Analysis
======================================================================
  038010_EIT-2A: Full agreement: 7, Partial (2/3): 10, No agreement: 13
  038011_EIT-1A: Full agreement: 4, Partial (2/3): 13, No agreement: 13
  038012_EIT-2A: Full agreement: 1, Partial (2/3): 5, No agreement: 24
  038015_EIT-1A: Full agreement: 6, Partial (2/3): 10, No agreement: 14

  Detailed results saved to /Users/rohithreddymandala/Documents/GitHub/autoe
it-gsoc-2026/output/evaluation/evaluation_results.json
  Summary saved to /Users/rohithreddymandala/Documents/GitHub/autoeit-gsoc-2
026/output/evaluation/evaluation_summary.json
  Consensus saved to /Users/rohithreddymandala/Documents/GitHub/autoeit-gsoc
-2026/output/evaluation/consensus_transcripts.json
```

The evaluation above shows **WER** (Word Error Rate), **CER** (Character Error Rate), **SER** (Sentence Error Rate), and **Exact** (number of sentences that perfectly match the stimulus) for each model on each participant file.

Key observations from the raw results:

- **OpenAI Whisper** has the lowest WER and most exact matches overall

- **CrisperWhisper** has slightly lower CER (character-level accuracy is marginally better)
- **Participant 038012** has dramatically higher error rates across all models — this reflects the participant's low proficiency, not model failure
- **AssemblyAI+Disfluencies** consistently performs worst, though it occasionally captures filler words ("uh", "um") that other models suppress

Below, we visualize the CER (our primary metric) in a structured comparison table:

In [15]:
```
# Evaluation summary table
summary_df = pd.DataFrame(summary)
pivot = summary_df.pivot(index="model", columns="audio_id", values="cer")
pivot["Average"] = pivot.mean(axis=1)
pivot = pivot.sort_values("Average")

print("CER (Character Error Rate) by Model and Participant\n")
print("Lower is better. CER measures character-level distance from the stimu
display(pivot.round(3))

print("\n\nOverall Model Ranking (by average CER):")
for i, (model, row) in enumerate(pivot.iterrows()):
    print(f"  {i+1}. {model:<30} CER: {row['Average']:.3f}")
```

```
CER (Character Error Rate) by Model and Participant

Lower is better. CER measures character-level distance from the stimulus.
```

audio_id model	038010_EIT-2A	038011_EIT-1A	038012_EIT-2A	038015_EIT-1A	Averag
crisper_whisper	0.300	0.314	0.498	0.202	0.32
openai	0.271	0.307	0.629	0.206	0.35
assemblyai_disfluencies	0.453	0.353	0.630	0.239	0.41

```
Overall Model Ranking (by average CER):
  1. crisper_whisper                CER: 0.329
  2. openai                         CER: 0.353
  3. assemblyai_disfluencies        CER: 0.419
```

Per-Participant Analysis

Participant 038012 shows significantly higher error rates across all models (CER ~0.5-0.6 vs ~0.2-0.3 for others). This participant exhibited very low Spanish proficiency — even manual listening was extremely difficult. This is not an ASR failure; it reflects genuine speaker difficulty with the EIT task.

Consensus Analysis

We also computed inter-model agreement using majority voting (ROVER-style):

- **Full agreement** (all 3 models identical): High confidence in transcription accuracy
- **Partial agreement** (2 of 3 match): Majority likely correct
- **No agreement** (all 3 differ): Most uncertain — may require manual review

In [16]:
```
# Consensus agreement breakdown
print("Inter-Model Agreement\n")
for audio_id in audio_ids:
    c = consensus[audio_id]
    full = sum(1 for x in c if x["agreement"] == "full")
    partial = sum(1 for x in c if x["agreement"] == "partial")
    none_ = sum(1 for x in c if x["agreement"] == "none")
    print(f"  {audio_id}:  Full: {full:2d}  |  Partial (2/3): {partial:2d}

total_full = sum(sum(1 for x in consensus[a] if x["agreement"] == "full") fo
total_partial = sum(sum(1 for x in consensus[a] if x["agreement"] == "partia
total_none = sum(sum(1 for x in consensus[a] if x["agreement"] == "none") fo
print(f"\n  Total:       Full: {total_full:2d}  |  Partial (2/3): {total_par
```

```
Inter-Model Agreement

  038010_EIT-2A:  Full:  7  |  Partial (2/3): 10  |  No agreement: 13
  038011_EIT-1A:  Full:  4  |  Partial (2/3): 13  |  No agreement: 13
  038012_EIT-2A:  Full:  1  |  Partial (2/3):  5  |  No agreement: 24
  038015_EIT-1A:  Full:  6  |  Partial (2/3): 10  |  No agreement: 14

  Total:       Full: 18  |  Partial (2/3): 38  |  No agreement: 64  (out of
120)
```

6. ASR Corrections

Per the task instructions: *"Be sure to correct only ASR errors, not participant grammar/vocabulary errors."*

Two ASR errors were identified and corrected:

File	Sentence	Original (OpenAI)	Issue	Correction Source
038015_EIT-1A	20	`[TRANSCRIPTION FAILED]`	Model returned no output	Re-ran OpenAI → `"El niño que se murió al gato y está triste."`
038012_EIT-2A	24	`"Pero yo ya me lo he gonfalonado..."` (hallucinated paragraph)	Whisper hallucination on low-proficiency speech	Used CrisperWhisper output: `"La cantina lay as a farsino las personas."`

How we distinguished ASR errors from participant errors:

- If all 3 models agree on a deviation from stimulus → participant actually said that (participant error, kept as-is)
- If 1 model produces gibberish/hallucination while others produce plausible Spanish → ASR error (corrected)
- If a model outputs text in the wrong language (English, Chinese) → ASR error (corrected)

7. Results

Final Output

- **120 transcriptions** (30 sentences × 4 participants) documented in `AutoEIT Sample Audio for Transcribing Updated.xlsx`, Column C (located in `data/` folder)
- **Primary model:** OpenAI Whisper — best overall performance (lowest WER: 0.479, most exact matches: 18/120, and competitive CER: 0.353)
- **2 ASR corrections** applied using alternative model outputs

Model Performance Summary

Model	Avg WER	Avg CER	Exact Matches	Best For
OpenAI Whisper	0.479	0.353	18/120	Overall accuracy, speed
CrisperWhisper	0.497	0.329	12/120	Character-level accuracy, occasional disfluency capture
AssemblyAI+Disfluencies	0.577	0.419	9/120	Disfluency detection ("uh", "um")

8. Challenges & Questions

Challenges Encountered

1. **Segmentation was the hardest problem.** Six different approaches failed before speaker voiceprint detection succeeded. The single-channel mixed recording (stimulus + participant on one mic) made traditional methods (diarization, amplitude thresholds, VAD) unreliable.

2. **No Spanish verbatim ASR model exists.** CrisperWhisper only supports English and German. All three models tested tend to "clean up" speech — removing disfluencies, correcting grammar — which is the opposite of what EIT transcription requires.

3. **No ground truth for evaluation.** Without human-generated reference transcriptions, we cannot compute true ASR accuracy. Our stimulus-based evaluation measures combined speaker deviation + ASR error.

4. **Participant proficiency varies dramatically.** Participant 038012 was extremely difficult for both human listeners and all ASR models (0% exact matches). The pipeline must handle the full proficiency spectrum.

5. **CrisperWhisper compatibility issues.** The `return_timestamps="word"` setting crashed on certain segments due to a known beam search bug. The `return_timestamps=True` setting caused inference to hang indefinitely. Only `return_timestamps=False` worked reliably.

Questions for Further Investigation

1. Would fine-tuning Whisper on Spanish conversational data (Fisher Spanish, CALLHOME Spanish) produce a viable verbatim model?
2. Can the voiceprint-based segmentation approach scale to hundreds of recordings without manual reference selection? Are there other segmentation approaches worth exploring?
3. Since our participants are non-native Spanish speakers (L2), their disfluency patterns may differ from native speakers (L1) — how should training data account for this?
4. What is the most effective way to recover disfluencies and filler words that standard ASR models suppress? (If fine-tuning a Spanish verbatim model proves viable per question 1, would a post-processing recovery pipeline still be worth exploring?)

9. Future Work — GSoC Project Directions

Based on the findings from this evaluation test, the following directions would address the key gaps identified:

1. Fine-tune Whisper for Spanish Verbatim Transcription

- Replicate CrisperWhisper's three-stage training methodology on Spanish data
- Training corpora: Fisher Spanish (~163h, LDC), CALLHOME Spanish (~20h, LDC), CIEMPIESS (~17h, open-source)
- Use LoRA fine-tuning for efficiency (single GPU, ~2.4M trainable parameters)
- Augment with synthetic disfluencies using disfluentES toolkit

2. Improve Segmentation Robustness

- Investigate audio fingerprinting if matched stimulus recordings become available
- Explore automated reference sample selection (removing the manual step)

- Test with larger participant pools to validate cross-speaker robustness

3. Build Ground Truth Dataset

- Manually transcribe a subset of recordings following the EIT protocol
- Use dual-annotator setup with inter-annotator agreement (Cohen's kappa)
- Enable proper evaluation with true WER/CER metrics

4. Post-Processing for Disfluency Recovery

- Implement gap detection pipeline (Whisper + wav2vec2 forced alignment)
- Classify gaps as disfluencies vs. silence
- Recover filler words that standard ASR suppresses

5. End-to-End Pipeline

- Integrate preprocessing, segmentation, transcription, and scoring into a single automated tool
- Web interface for researchers to upload EIT recordings and receive transcriptions
- Support for multiple languages (the EIT exists in English, German, Japanese, French, Korean, Portuguese, Mandarin)

Repository Structure

```
autoeit-gsoc-2026/
├── data/
│   ├── raw/                          # Original MP3
recordings
│   ├── trimmed/                      # Preprocessed WAV/MP3
files
│   ├── references/                   # Speaker reference
samples (3-5s each)
│   ├── participant_segments/         # 30 individual
response clips per participant
│   ├── AutoEIT Sample Audio for Transcribing.xlsx         #
Original test file
│   └── AutoEIT Sample Audio for Transcribing Updated.xlsx  #
Final output with transcriptions in Column C
├── src/
│   ├── preprocess.py                 # Audio trimming
│   ├── detect_speaker.py             # SpeechBrain ECAPA-
TDNN speaker voiceprint detection
│   ├── export_segments.py            # Export individual
response clips
│   ├── transcribe.py                 # CrisperWhisper
transcription
```

```
│   ├── transcribe_cloud.py          # OpenAI Whisper +
AssemblyAI transcription
│   ├── evaluate.py                  # CER/WER/SER
evaluation
│   ├── export_excel.py              # Export transcriptions
to Excel
│   ├── retry_segments.py            # Retry failed segments
with OpenAI
│   └── analyze_tone.py              # Tone detection
analysis (exploratory)
├── tests/
│   └── test_preprocess.py           # Unit tests for
preprocessing
├── notebooks/
│   └── autoeit_transcription.ipynb   # This notebook
├── output/
│   ├── transcriptions/              # JSON outputs per
model (openai, crisper_whisper, assemblyai_disfluencies)
│   ├── speaker_detection/           # Detection results
(JSON) and similarity plots (PNG)
│   └── evaluation/                  # Evaluation metrics
and consensus transcripts
├── requirements.txt
└── .env.example
```